CS 301 · MEETING 14 · ACT 2

# Two Products

A matrix times its transpose, in two orders: one compares the rows with one another, the other the columns.

# A matrix times its transpose, two ways

$$(XX^T)_{ij} = \text{row}_i \cdot \text{row}_j \qquad (X^T X)_{ij} = \text{column}_i \cdot \text{column}_j$$

**X with n rows and m columns**

| product | shape | compares | multiplications |
|---|---|---|---|
| `X @ X.T` | n × n | the rows: the people | n · n · m |
| `X.T @ X` | m × m | the columns: the genres | m · m · n |

- Both symmetric: cell (i, j) equals cell (j, i)

# Six people, four genres

```
X = (Bf @ G)[selected rows, four genres]: how many movies of each genre each person rated
            Action  Comedy  Drama  Animation
person 342       9      24      9          2
person 228      12       2      6          2
person 350      14      16      8          3
person 231      11       4      9          0
person 341      12      27     10          4
person 48       22       8      8          3

X.T @ X, genres by genres: the dot product of two columns
           Action  Comedy  Drama  Animation
Action       1170    1008    660        198
Comedy       1008    1645    726        232
Drama         660     726    426        118
Animation     198     232    118         42

X @ X.T is 6 by 6, people by people: 6 x 6 x 4 = 144 multiplications; X.T @ X: 4 x 4 x 6 = 96
```

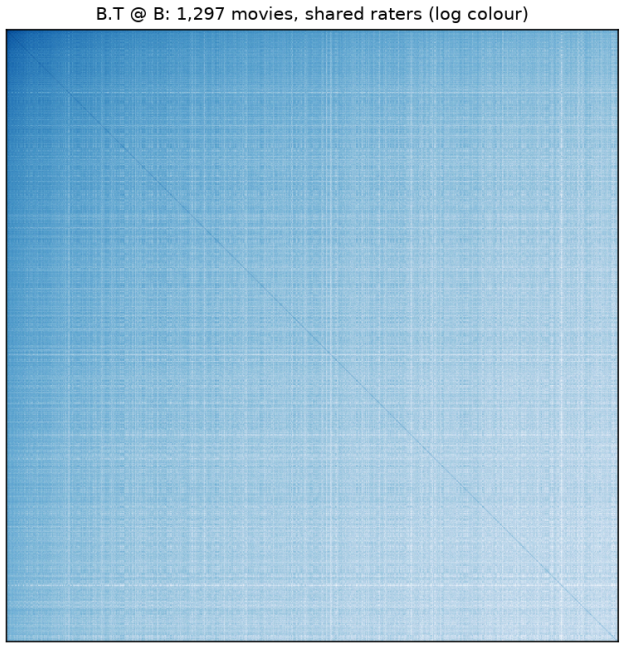
# Counts from the rating mask

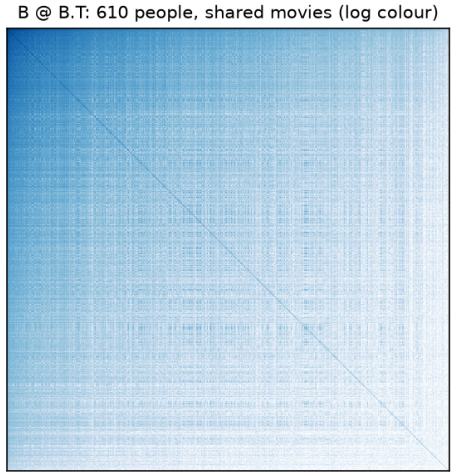
```
Bf = B.astype(float)
shared_raters = Bf.T @ Bf      # movies by movies: the people who rated both
shared_movies = Bf @ Bf.T      # people by people: the movies both rated
```

```
B.T @ B is 1,297 by 1,297: 1.03 billion multiply-adds, 0.01 s
B @ B.T is 610 by 610: 0.48 billion multiply-adds, 0.00 s
```

- `B.T @ B`: cell (i, j) counts the people who rated both movie i and movie j
- `B @ B.T`: the movies that both person i and person j rated
- $1{,}297^2 \times 610 \approx 1.03$ billion scalar multiplications; timings shown above

# Shared raters, shared movies




```
the pair of movies with the most raters in common: The Shawshank Redemption and Forrest Gump, 231
people
the pair of people with the most movies in common: 414 and 599, 854 movies
```

# Cosine similarity

$$\cos(u, v) = \frac{u \cdot v}{\|u\| \, \|v\|}, \qquad S = U^T U \text{ with every column of } U \text{ of length } 1$$

```
def cosine_of_columns(M):
    lengths = np.linalg.norm(M, axis=0, keepdims=True)
    if np.any(lengths == 0):
        raise ValueError("Cosine similarity is undefined for a zero-length column.")
    U = M / lengths
    return U.T @ U
```

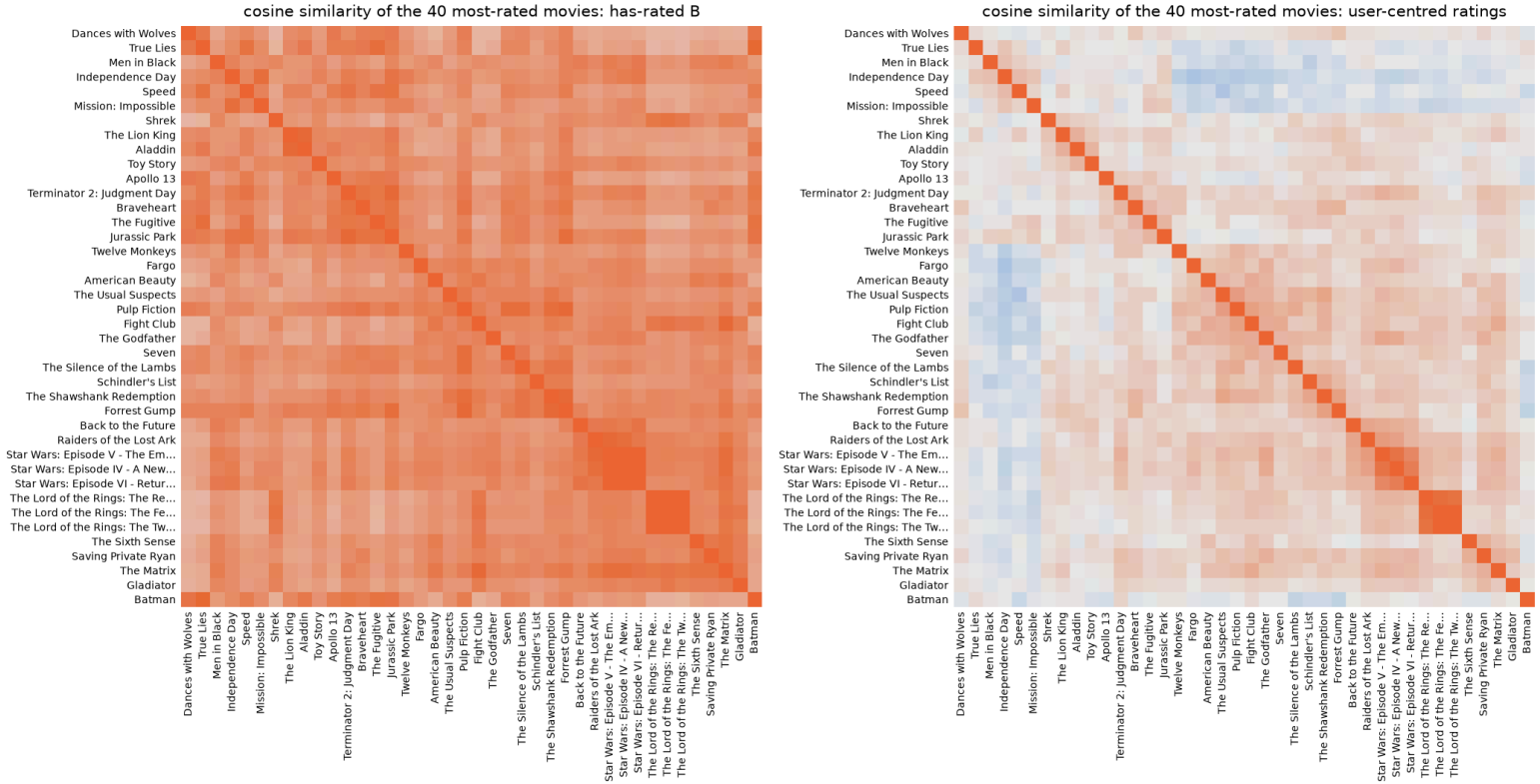
- Meeting 10 compared word vectors by the cosine: divide by the lengths, then the dot product
- Divide every column by its length, and all the cosines at once are one product

# Similarity under different rating matrices

| the matrix | a cell holds | what cosine compares |
|---|---|---|
| `B`, has-rated | 1 if the person rated the movie | rated by the same people |
| `R` with 0 in the holes | observed stars, with 0 for missing entries | high stars from the same people |
| `Rc`, user-centred | stars minus the person's mean; 0 in the holes | above or below their usual, together |

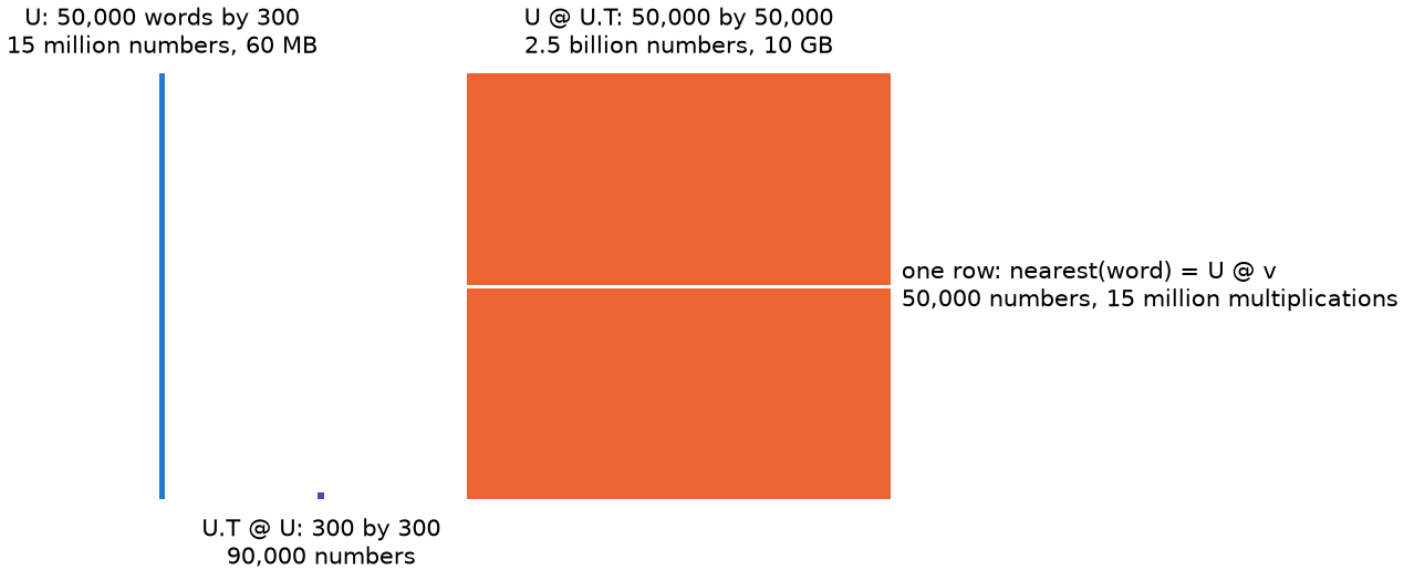
```
          has-rated B:  ['Independence Day', 'Jurassic Park', 'Forrest Gump', 'Star Wars: Episode IV
- A New Hope', 'Mission: Impossible', 'Star Wars: Episode VI - Return of the Jedi', 'Toy Story 2']
     ratings as given:  ['Toy Story 2', 'Jurassic Park', 'Independence Day', 'Star Wars: Episode IV
- A New Hope', 'Forrest Gump', 'The Lion King', 'Star Wars: Episode VI - Return of the Jedi']
  user-centred ratings:  ['Toy Story 2', 'Aladdin', 'Toy Story 3', 'Wallace & Gromit: The Wrong
Trousers', 'The Incredibles', 'Finding Nemo', 'Blazing Saddles']
```

# Similarities among 40 movies




- Under `B`: every cell positive; popular movies share raters; the series have especially high similarities
- Under `Rc`: blocks, and blue cells where the same people rate in opposite directions

# Products of the word-vector matrix




- Meeting 10: `nearest(word)` = `U @ v`, one row of `U @ U.T`: 15 million multiplications
- The full product has 2.5 billion cells: 10 GB in float32 for the output alone
- `U.T @ U`, 300 by 300: the dimensions compared with one another

# Correlation, similarity and PCA of word vectors

- Are the 300 dimensions correlated with one another? `Z.T @ Z / n`, with n = 50,000 words
- Which words are similar? `U @ U.T`, one row at a time
- Along which directions is the cloud of 50,000 words spread out? Act 3